CYCLAW · DOCS / ! HOW-TO-GUIDES

NetConnect How-To

Passive, default-off LAN inventory for a host you administer.
v0.1.0 · `agentic/netconnect/` · PR #1168 + #1189 + #1239

Verified against origin/main HEAD `ccf63b03118556bcbfe608f12eb38b0bcb3b7183` (2026-09-06). Live files: `cli.py client.py config.py context.py scope.py selftest.py`. Graph still 12-node. I6 isolation unchanged. Not a seventh out-of-band root package.

> **READ THIS FIRST**
>
> This is not a scanner, not a mapper, not nmap, not ping, not a discovery daemon. It reads local hostname/interface names and the OS neighbor cache that already exists. Enable it only on a network you administer. Leave it off the rest of the time. The audit event name is `netconnect_scan` — that name is historical and does not license an active probe later.

Contents

1. What it is, and what it is not

NetConnect is a Phase-2 foundation module that answers one operator question: *what does this machine already know about the local Layer-2 neighborhood?* It does that with two ops only.

Op	What it actually does	What it does not do
`self`	Best-effort local identity: `socket.gethostname()` plus `socket.if_nameindex()`. Returns interface index + name. `addresses` is always an empty list. `complete: false`. `active_probe: false`.	No DNS. No getaddrinfo. No socket connect. No route table. No address enumeration. Tests assert this.
`arp`	Runs a fixed absolute OS command with `shell=False` and `command_timeout_sec` (ships 5s). Parses the existing neighbor cache. Filters to IPv4 inside `allowed_cidrs`. Sorts. Truncates at `max_neighbors` (ships 512).	No ARP request. No ping. No port probe. No subnet sweep. No packet the kernel was not already holding.

Two more CLI verbs exist and they never touch the network: `status` prints the live config (and unknown keys) and `test` runs the offline self-test against parser fixtures. Use those first.

> **PASSIVE IS PACKET PHYSICS, NOT INNOCENCE**
>
> The neighbor cache is the residue of prior L2 traffic. Enabling `arp` is reading a ledger of who recently talked on the LAN. Stale is not harmless. Results are hints, never reachability, never completeness. `source` on arp output is literally `existing_neighbor_cache`. Treat a quiet cache as 'we don't know', not 'the LAN is empty'.

2. Where it sits in the architecture

Package path: `agentic/netconnect/`. Files: `__init__.py cli.py client.py config.py context.py scope.py selftest.py`. It lives under the existing `agentic/` out-of-band package the same way fsconnect and sqlconnect do. It is **not** a seventh top-level OOB package. OUT_OF_BAND_PKGS stays six: agentic / sync / guardrails / harness / telegram / opentweet.

I6 isolation: `gate.py`, `graph.py`, and `mcp_hybrid_server.py` must never import this package. There is no `/query` route, no `/ops/netconnect` shim, no harness slash command, no scheduler, no Telegram hook, no UI panel. If you need inventory, you run the CLI on the box. That is the whole interface.

Logging: #1239 wired `setup_logging` on all four agentic CLI entrypoints including this one, so warnings reach `logs/cyclaw.log`. A broken `logging:` block or unwritable log file is exit 3, not a traceback exit 1.

3. Config contract

Top-level block in `config.yaml`. Missing block, wrong types, or `enabled: true` with an empty CIDR list all raise `NetConnectConfigError` and the CLI exits 3. `enabled` must be a real YAML boolean. Quoted `"true"` is not True.

```
# Passive LAN inventory connector (out-of-band)  [v0.1]
# Reads only local host metadata and the OS existing neighbor cache.
# No ping, port probe, subnet sweep, packet send, scheduler, or API route.
netconnect:
  enabled: false
  allowed_cidrs: []              # e.g. ["192.168.1.0/24"]; required when enabled
  allowed_net_ops:
    - self
    - arp
  command_timeout_sec: 5
  max_neighbors: 512
```

Key	Shipped	Bounds / fail-closed
`enabled`	false	Literal bool. False = self/arp print a no-op heading and exit 0.
`allowed_cidrs`	[]	Non-empty list of IPv4 CIDR strings required when enabled. See §4.
`allowed_net_ops`	[self, arp]	Must be a subset of {self, arp}. Unknown op → config error.
`command_timeout_sec`	5	Positive float, ceiling 30. Converted to float.
`max_neighbors`	512	Integer 1..4096. Truncation sets truncated: true.

Unknown keys are not fatal. They land on `_unknown_keys` and `status` prints them to stderr (#1189). A typo like `allowed_cdirs` will silently keep the empty default and then fail closed when you flip enabled. Run status after every edit. Do not conflate the shipped default (`allowed_cidrs: []`) with the README enablement example (`["192.168.1.0/24"]`). That /24 is an EXAMPLE, not a dump of your LAN and not TEST-NET-1 (`192.0.2.0/24` is public and `scope.py` refuses it).

4. Scope rules — including the hole the PR copy oversold

Allowed parent networks in `scope.py`:

```
ALLOWED_PARENT_CIDRS = (
    "10.0.0.0/8",
    "172.16.0.0/12",
    "192.168.0.0/16",
    "127.0.0.0/8",
)
```

Each configured CIDR is parsed with `ip_network(..., strict=False)`, must be IPv4, must satisfy `network.subnet_of(parent)` for one of those four parents, then is stored as `with_prefixlen` and deduped. Rejects: public space, CGNAT `100.64.0.0/10`, link-local `169.254.0.0/16`, IPv6, garbage syntax.

CODE DOES NOT ENFORCE A MINIMUM PREFIX LENGTH

PR #1168 prose said “overly broad scopes” fail closed. Live code only requires the CIDR to be a subnet of a parent. In Python, a network is a subnet of itself, so `10.0.0.0/8` is accepted. There is no min-prefix check. If you write `allowed_cidrs: ["10.0.0.0/8"]` you just authorized inventory across sixteen million addresses the host might have seen. Use the actual /24 (or tighter) you administer. The code will not save you from being sloppy.

Recommended operator values: the LAN the Mac or workstation is actually on. Example: `["192.168.1.0/24"]`. Add loopback only if you need it (`127.0.0.0/8`). Do not paste every RFC1918 parent “just in case.”

IPv6 is a known blind spot, not a feature. Dual-stack LANs will look half-empty. v0.1 drops non-v4 records and refuses IPv6 CIDRs. Do not conclude the LAN is inventoried.

5. Operator sequence

Do this in order. Skipping status/test is how you get a quiet success that collected nothing.

1. status — always safe
`python -m agentic.netconnect.cli status`
Read enabled, CIDRs, ops, timeout, max_neighbors, and the hard-coded `active_scanning: False`. Watch stderr for unknown keys.

2. test — offline preflight
`python -m agentic.netconnect.cli test`
Four fixture checks. Never runs the platform command. Must print N/N passed.

3. edit config
Set `enabled: true` and the narrowest CIDR you actually administer. Save. Do not commit a household CIDR into a public fork if that identifies your LAN.

4. status again
Confirm the CIDR list is the one you typed, enabled is the boolean True, active_scanning is still False, stderr is clean.

5. self, then arp
Capture JSON to a file you control. Do not pipe into RAG, Telegram, tickets, or paste buffers.

6. treat JSON as hints
Missing neighbor ≠ down host. Present neighbor ≠ reachable service. `complete: false` on self is the contract.

7. disable when idle
Flip `enabled: false`. Default-off is the privacy posture. Leave it that way.

AUTHORIZATION BOUNDARY — ENABLE ONLY IF ALL FIVE ARE TRUE

1. You control the CyClaw host. 2. You are authorized to administer every IPv4 subnet in allowed_cidrs. 3. The CIDR is the narrowest net you actually administer (code WILL accept 10.0.0.0/8; that is not permission). 4. You are not on guest / hotel / client / coffee-shop / neighbor Wi-Fi. 5. Corporate or school AUP does not forbid local network-inventory tooling.
If any item is false: leave `enabled: false`. Advisory only. Not an authorization to inventory a network you do not administer.

6. CLI reference

```
python -m agentic.netconnect.cli [--config config.yaml] {status|self|arp|test}
```

Verb	Does	When disabled
`status`	Prints config + active_scanning: False. Unknown keys on stderr.	Still runs. Always safe.
`self`	Local hostname + interface names as JSON.	Prints “Network connector disabled” and exits 0.
`arp`	Scoped neighbor-cache JSON.	Same no-op, exit 0.
`test`	Offline self-test (config load, public CIDR refuse, scope, parser fixtures).	Runs regardless of enabled.

Exit codes

Code	Name	Meaning
`0`	EXIT_OK	Success, including the disabled no-op on self/arp. Wrappers that treat 0 as “inventory collected” will lie. Require the JSON key op first.
`2`	EXIT_FAIL	NetConnectError at runtime, or self-test failed a fixture.
`3`	EXIT_ENV	Config error, logging init failure, or missing platform binary (arp.exe / ip / /usr/sbin/arp).

Platform commands (fixed argv, shell=False)

The OS command dumps the whole cache. Scope filtering happens in-process after parse. That is intentional and it is a residual: a debug dump of raw stdout, a compromised process, or a future “log the raw cache” change would leak out-of-scope neighbors. Windows needed an extra filter (`_scope_interface`) because `arp.exe -a` puts the local interface IPv4 in the interface field.

OS	Resolved command
`Windows`	%SystemRoot%\System32\arp.exe -a (fallback r"C:\Windows")
`Linux`	first existing of /usr/sbin/ip, /sbin/ip, /usr/bin/ip, /bin/ip + neigh show
`Darwin`	/usr/sbin/arp -an
`anything else`	NetConnectRuntimeError — unsupported platform

7. Output fields and audit surface

self JSON (stdout — not redacted)

```
{
  "op": "self",
  "hostname": "macbook-pro.local",
  "interfaces": [{"index": 1, "name": "lo0"}, {"index": 14, "name": "en0"}],
  "addresses": [],
  "complete": false,
  "active_probe": false,
  "limitations": "...no address/route enumeration...",
  "errors": []
}
```

arp JSON (stdout — contains IP and MAC; handle as personal data)

```
{
  "op": "arp",
  "platform": "Darwin",
  "source": "existing_neighbor_cache",
  "active_probe": false,
  "observed_count": 17,
  "in_scope_count": 9,
  "truncated": false,
  "neighbors": [
    {"ip": "192.168.1.1", "mac": "aa:bb:cc:dd:ee:ff", "interface": "en0", "state": "cached"}
  ]
}
```

Audit event: `netconnect_scan`. Payload is scope hashes (`hash_query(cidr)`) plus aggregate counts (interface_count / observed_count / in_scope_count / result_count / truncated / error_count). Tests assert raw IP and MAC are absent from `audit.jsonl`. Numbat maps the event to `network.indicator`. IR, EDR, and a future reviewer will read the word “scan.” Document the behavior as cache-read every time you talk about the event name. Do not let the name license an active probe later.

STDOUT IS NOT THE AUDIT TRAIL

CLI JSON to the terminal includes hostname, interface names, IPv4, and MAC. That stream is personal data in household and many workplace contexts. Do not redirect it into the RAG corpus, soul.md, memory notes, Telegram, OpenTweet, or a ticket system. If you must keep a snapshot, keep it off-repo, mode 600, and add it to the data map.

8. Privacy and legal — advisory, not counsel

This section is operator hygiene for Christopher Grady / CGFixIT. It is not a legal opinion, not a privilege letter, and not a substitute for counsel if this box sits on a network you do not own.

Regime	Why it matters here
`GDPR Art. 4(1) / Recital 30`	IP + MAC + hostname can single out a device and, on a household LAN, a person. When enabled, add this surface to the data map next to the auth DB, memory DB, audit.jsonl, and the Numbat stream. Art. 25 privacy-by-design is already the shipped posture: default-off, CIDR gate, audit hashes not values. Do not wreck that by persisting arp JSON.
`CCPA / CPRA`	IP address is commonly treated as personal information. Same handling rule: don't persist, don't share.
`CFAA 18 USC 1030`	Reading the local OS neighbor cache on a host you control is not, by itself, unauthorized access to a third-party host. Federal unauthorized-access risk is low for that fact pattern; it is not a defense to AUP, employment, or client-site rules. An active sweep changes the analysis. v0.1 has no sweep and must not grow one without a fresh review.
`Georgia O.C.G.A. § 16-9-93`	Computer trespass / invasion of privacy. Same split as CFAA for an Atlanta operator: own-host cache read vs. touching systems you do not administer.
`Wiretap Act`	An ARP/neighbor cache is not intercept of communications in transit. Do not describe this module as “listening to the LAN.” It is reading a kernel table.
`Corporate / guest / HOA Wi-Fi`	Authorization and acceptable-use still apply to passive inventory. If you did not provision the network, do not enable this. Home lab you own: fine. Airport / client office / apartment building: no.
`DSR / deletion`	Persisted arp JSON is in-scope for a deletion request. CIDR hashes in audit.jsonl are not raw identifiers. Don't create a third copy.

9. Residual risks and hard no's

These are the things the how-to is for. Do not paper over them in a later revision.

- **Audit name is a tell.** `netconnect_scan` will be read as scanning. Behavior is cache-read. Keep saying so.
- **Windows filter is in-process only.** The OS command sees everything. Python filters after. Don't log raw command output.
- **Disabled CLI exits 0.** Automation that keys off exit codes will invent a successful inventory. Parse JSON.
- **IPv6 is invisible.** Dual-stack inventory is incomplete by construction in v0.1.
- **/8 and /12 are legal config.** Narrow the CIDR yourself. See §4.
- **No /ops, no harness, no Telegram, no scheduler, no /query.** If someone adds one of those, this how-to is withdrawn until re-reviewed.
- **No DNS, no getaddrinfo, no socket connect, no ping, no nmap, no port probe, no packet send.** A one-line "just resolve the hostname" PR is a phase change, not a polish pass.
- **Do not ingest arp JSON into RAG.** That turns a local cache read into a durable personal-data store inside the soul/corpus path.

> **WHAT THIS MUST NEVER BECOME**
>
> NetConnect v0.1 is a cache reader with a CIDR gate. It is not a reconnaissance tool, not an asset-management platform, and not an excuse to put LAN topology in the request path. Future phases that send packets need a new threat-model amendment, a new how-to, and an explicit operator decision — not a quiet flag.

10. Troubleshooting and self-test

Symptom	Likely cause	Fix
`self/arp print "Network connector disabled" and exit 0`	enabled is false (shipped default) or not a literal bool	status first. Set enabled: true as a YAML bool. Do not quote it.
`exit 3 on enable`	allowed_cidrs empty, public/CGNAT/v6 CIDR, unknown op, bad timeout	status + read the Config error line. Narrow to RFC1918/loopback.
`stderr: unknown netconnect keys`	typo in config.yaml (#1189)	Fix the key. status should go quiet.
`exit 3, command not installed`	arp.exe / ip / /usr/sbin/arp missing from the fixed path list	Install the OS networking tools. Don't invent a PATH search.
`arp JSON empty on a live LAN`	Wrong CIDR, quiet cache, IPv6-only neighbors, or scope filter ate them	Check allowed_cidrs vs the actual iface. Remember v0.1 is IPv4-only. Empty ≠ down.
`truncated: true`	in-scope neighbors exceeded max_neighbors	Raise toward 4096 only if you have a reason. Prefer a tighter CIDR.
`test is not N/N`	config.yaml drifted or parser fixture mismatch	Do not enable. Diff config against shipped block. Re-run test.

Self-test coverage (offline — never runs the platform command)

01 config loads 02 public CIDR refused 03 scope policy 04 Windows / Linux / Darwin parser fixtures.

```
python -m agentic.netconnect.cli test
python -m agentic.netconnect.cli status
python -m agentic.netconnect.cli self
python -m agentic.netconnect.cli arp
```

References

- Live module: `agentic/netconnect/` on origin/main.
- Shipped config: top-level `netconnect:` block in `config.yaml`.
- PRs: #1168 (feature), #1189 (unknown keys on status + Windows SystemRoot fallback), #1239 (setup_logging on the CLI).
- Advisor stamp: cyclaw-advisor against HEAD ccf63b03 (2026-09-06). 12-node graph, invariant-guard 47/47, OOB six.
- Sibling how-tos live under `docs/! How-To-Guides/`. Drop this PDF there if you commit it.
- This document does not amend THREAT_MODEL.md. A future active-probe phase would.

Generated 2026-09-06 for C.G. / CGFixIT. Unfiltered operator guide. Re-verify against origin/main before treating any command as current if the tree has moved.